

Frontend onboarding course

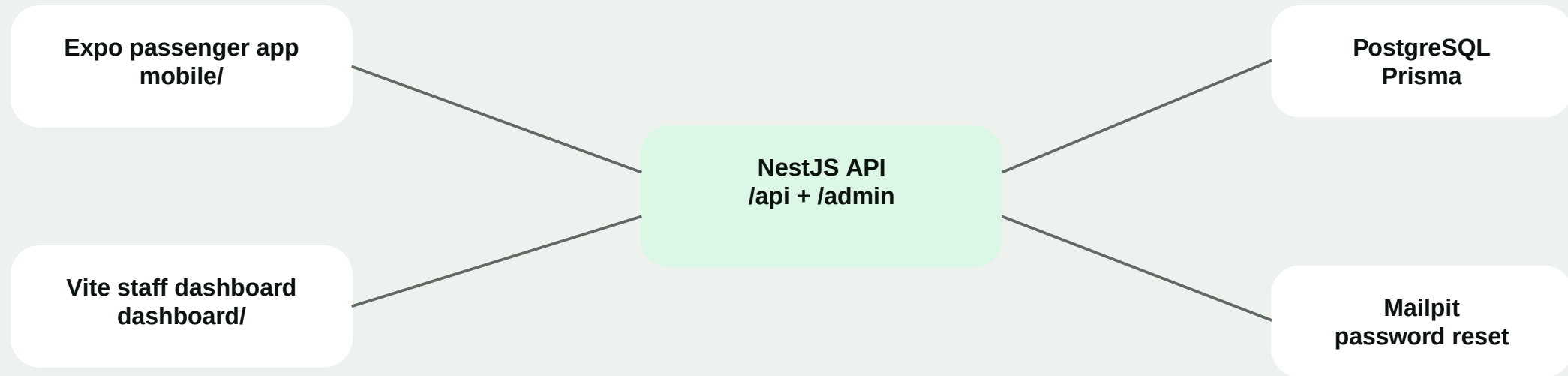
Expo passenger app + React/Vite operations dashboard
Real code, demos, debugging and code review

NEW EMPLOYEES

FRONTEND

RailPass system map

One API, two frontends, one design language



- Frontend lessons reference backend contracts where they shape UX.
- Design tokens mirror across mobile constants and dashboard CSS variables.
- Server owns pricing and seat-concurrency truth.

Course paths

Choose the schedule that fits the cohort

1-day intensive

- Map
- Tokens
- State
- API failure
- Review

2-day course

- Day 1: lessons 1–4
- Day 2: lessons 5–8
- Mini PR review

4 half-days

- Map + tokens
- Flow + state
- API + debugging
- Quality + extension

Course rule: token-first UI

No hard-coded design values in components

```
// Do
row: { borderRadius: radii.xl, padding: spacing.md }

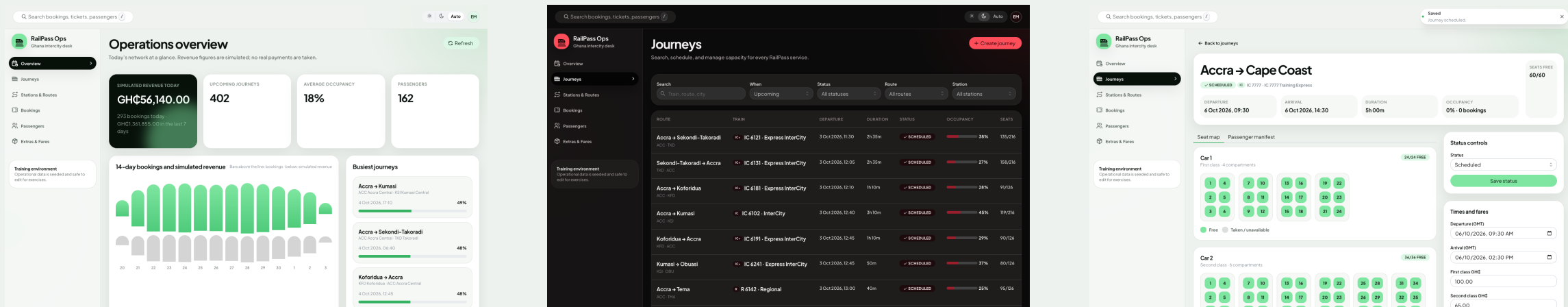
// Don't
row: { borderRadius: 24, padding: 12, backgroundColor: '#FF4D5A' }
```

Why it matters

- Themes stay aligned.
- Danger and red brand remain distinct.
- Screenshots are predictable.
- Reviews focus on intent.

Screenshots as review artifacts

Use visuals to discuss tokens, states and layout



Mobile screenshots were not present during generation; use live Expo demos for mobile flows.

Lesson 01

Project organisation

60 min

- Thin Expo Router entries
- Feature boundaries
- Naming/import order
- Constants vs config

Lesson 01: objectives

Project organisation

- Thin Expo Router entries
- Feature boundaries
- Naming/import order
- Constants vs config

Code paths

Open the lesson notes for exact files and demos.

Lesson 01: decisions and trade-offs

What we chose, why, and when to choose differently

Routes delegate

Config separated

Dashboard guard once

Review prompt: what would make us choose differently?

Lesson 01: anchor snippet

Short, real excerpts from the monorepo

```
import { SignInScreen } from '@features/auth/screens/sign-in-screen';  
export default SignInScreen;
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 01: activity

Interactive practice

Refactor a route that mixes fetching, rendering and styling.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Components and design tokens

75 min

- Use tokens everywhere
- Light/dark palettes
- Danger separate from red brand
- Extend components safely

Lesson 02: objectives

Components and design tokens

- Use tokens everywhere
- Light/dark palettes
- Danger separate from red brand
- Extend components safely

Code paths

Open the lesson notes for exact files and demos.

Lesson 02: decisions and trade-offs

What we chose, why, and when to choose differently

StyleSheet + constants

Mantine + CSS variables

Danger token, not brand red

Review prompt: what would make us choose differently?

Lesson 02: anchor snippet

Short, real excerpts from the monorepo

```
danger: {  
  background: colors.dangerSoft,  
  border: colors.danger,  
  tone: 'danger',  
}
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 02: activity

Interactive practice

Replace hard-coded styles in a flawed example with tokens.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 03

Navigation and flows

75 min

- Trace one-way/round-trip booking
- Explain Stack.Protected
- Use modal station picker
- Trace dashboard routes

Lesson 03: objectives

Navigation and flows

- Trace one-way/round-trip booking
- Explain Stack.Protected
- Use modal station picker
- Trace dashboard routes

Code paths

Open the lesson notes for exact files and demos.

Lesson 03: decisions and trade-offs

What we chose, why, and when to choose differently

Protected groups

Modal station picker

Route params for minimal context

Review prompt: what would make us choose differently?

Lesson 03: anchor snippet

Short, real excerpts from the monorepo

```
<Stack.Screen name="station-picker" options={{ presentation: 'modal' }} />  
<Stack.Screen name="journeys/[id]" />  
<Stack.Screen name="checkout" />
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 03: activity

Interactive practice

Draw the booking flow and mark route state vs draft store.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 04

State management

90 min

- Classify client vs server state
- Use persist/partialize
- Use selectors/useShallow
- Invalidate query caches

Lesson 04: objectives

State management

- Classify client vs server state
- Use persist/partialize
- Use selectors/useShallow
- Invalidate query caches

Code paths

Open the lesson notes for exact files and demos.

Lesson 04: decisions and trade-offs

What we chose, why, and when to choose differently

Zustand for mobile draft

useApiQuery for modest mobile server state

TanStack Query for dashboard

Review prompt: what would make us choose differently?

Lesson 04: anchor snippet

Short, real excerpts from the monorepo

```
partialize: ({ origin, destination }) => ({ origin, destination })  
queryClient.invalidateQueries({ queryKey: ['journeys'] });
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 04: activity

Interactive practice

Classify ten values as route, component, client, server or derived state.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 05

API, forms and failures

90 min

- Normalize API errors
- Map field errors
- Trust server quote
- Recover from SEAT_TAKEN

Lesson 05: objectives

API, forms and failures

- Normalize API errors
- Map field errors
- Trust server quote
- Recover from SEAT_TAKEN

Code paths

Open the lesson notes for exact files and demos.

Lesson 05: decisions and trade-offs

What we chose, why, and when to choose differently

One Axios client

Server quote is source of truth

Conflict recovery is normal

Review prompt: what would make us choose differently?

Lesson 05: anchor snippet

Short, real excerpts from the monorepo

```
const quote = useApiQuery(  
  request ? `quote:${JSON.stringify(request)}` : null,  
  () => bookingsApi.quote(request!),  
);
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 05: activity

Interactive practice

Trigger invalid email and trace fieldErrors to TextField error.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 06

Performance and debugging

90 min

- FlatList/DataTable patterns
- Skia + native Pressables
- Reduced motion
- Debug stale state maps

Lesson 06: objectives

Performance and debugging

- FlatList/DataTable patterns
- Skia + native Pressables
- Reduced motion
- Debug stale seat maps

Code paths

Open the lesson notes for exact files and demos.

Lesson 06: decisions and trade-offs

What we chose, why, and when to choose differently

Skia draws; native controls interact

Focus-scoped polling

Memoized taken-seat sets

Review prompt: what would make us choose differently?

Lesson 06: anchor snippet

Short, real excerpts from the monorepo

```
const taken = useMemo(() => new Set(car.takenSeats), [car.takenSeats]);  
const reduceMotion = useReducedMotion();
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 06: activity

Interactive practice

Debug a selected seat that becomes taken while the screen is open.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 07

Accessibility, testing, review

75 min

- Roles/labels/states
- Test visible behavior
- Husky hooks
- Review checklist

Lesson 07: objectives

Accessibility, testing, review

- Roles/labels/states
- Test visible behavior
- Husky hooks
- Review checklist

Code paths

Open the lesson notes for exact files and demos.

Lesson 07: decisions and trade-offs

What we chose, why, and when to choose differently

Icon + label + colour

Targeted tests

Backend e2e proves concurrency

Review prompt: what would make us choose differently?

Lesson 07: anchor snippet

Short, real excerpts from the monorepo

```
expect(await screen.findByLabelText('Delayed +25 min')).toBeTruthy();  
expect(statuses).toEqual([201, 409]);
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 07: activity

Interactive practice

Write five actionable review comments for flawed examples.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Lesson 08

Extending maintainably

75 min

- Plan across layers
- Know when to abstract
- Avoid duplicate responsibilities
- Write acceptance criteria

Lesson 08: objectives

Extending maintainably

- Plan across layers
- Know when to abstract
- Avoid duplicate responsibilities
- Write acceptance criteria

Code paths

Open the lesson notes for exact files and demos.

Lesson 08: decisions and trade-offs

What we chose, why, and when to choose differently

Feature-local first

Inputs in query keys

Backend DTO contract

Review prompt: what would make us choose differently?

Lesson 08: anchor snippet

Short, real excerpts from the monorepo

```
const key = originId && destinationId
  ? `journeys:${originId}:${destinationId}:${date}:${sort}:${passengers}`
  : null;
```

Ask learners

- Which layer owns this?
- What invariant does it protect?
- What bug appears if moved?
- What verification covers it?

Lesson 08: activity

Interactive practice

Plan a first-class-only or departs-after-noon filter end-to-end.

- Prediction first.
- Debugging task.
- Code review.
- Implementation challenge with acceptance criteria.

Extension A: Tickets and PDF417

Optional extension module

- API generates PDF417 matrix.
- Mobile draws one Skia path.
- PDF uses same matrix as SVG.
- Black-on-white stays scannable.

```
export function pdf417Matrix(payload: string):  
BarcodeMatrix {  
  return { format: 'PDF417', payload, columns, rows };  
}
```

Extension B: Dashboard operations

Optional extension module

- Redux owns auth/UI only.
- TanStack Query owns server data.
- Mantine components use tokens.
- CSS Modules use --rp-* variables.

```
const journeys = useJourneys({ page, pageSize,  
  search });  
const density = useAppSelector((s) =>  
  s.ui.tableDensity);
```

Extension C: API contracts

Optional extension module

- Backend error shape drives forms.
- Server pricing is authoritative.
- Unique index prevents races.
- E2E proves one 201 and one 409.

```
@@unique([journeyId, carNumber, seatNumber])  
expect(statuses).toEqual([201, 409]);
```

Final review checklist

- Paths and ownership are correct.
- Design values come from tokens.
- Server state is not duplicated.
- Errors and conflicts have recovery paths.
- Accessible labels/roles/states exist.
- Tests or verification evidence are included.

**Ship small
Review deeply
Learn the code**